Object-Oriented Programming Report

Oct 16, 2026 · @Alibi Auyespek

1. Введение

Объектно-ориентированное программирование (ООП) строит программу из объектов. Каждый объект хранит свои данные и умеет выполнять действия над ними. В основе ООП лежат четыре принципа: абстракция, инкапсуляция, наследование и полиморфизм.

Цель работы: показать все четыре принципа на одном небольшом примере на Python. Пример — транспортные средства: общий абстрактный класс `Vehicle` и два конкретных вида транспорта, `Bike` (велосипед) и `Car` (машина).

Технологии: Python 3, модуль `abc` (Abstract Base Classes).

2. Класс и объект

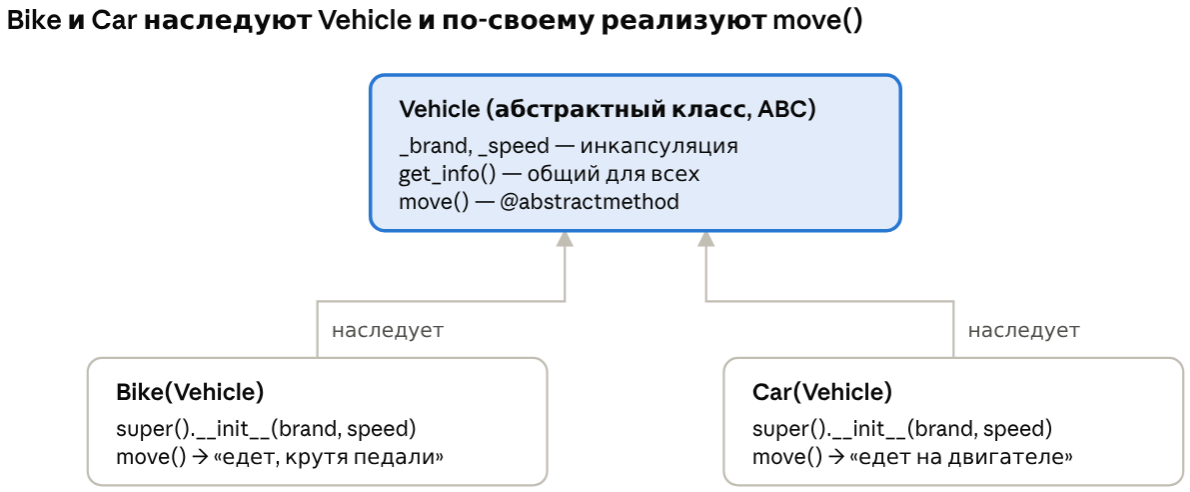
Класс — это чертёж: он описывает, какие данные (атрибуты) и какие действия (методы) будут у объектов. **Объект** — конкретная вещь, созданная по этому чертежу.

```
trek = Bike("Trek", 25)        # объект класса Bike
toyota = Car("Toyota", 180)   # объект класса Car
```

Понятие	Что это	В нашем коде
Класс	шаблон для создания объектов	`Vehicle`, `Bike`, `Car`
Объект	экземпляр класса	`Bike("Trek", 25)`
Атрибут	данные объекта	`_brand`, `_speed`
Метод	функция внутри класса	`get_info()`, `move()`
Конструктор	метод, который запускается при создании объекта	`__init__(self, brand, speed)`
`self`	ссылка на текущий объект	`self._brand = brand`

3. Четыре принципа ООП

Все четыре принципа работают в одной иерархии классов: `Vehicle` задаёт общий шаблон, а `Bike` и `Car` его дополняют.



иерархия классов · 1 абстрактный класс, 2 наследника

Стрелки идут от наследника к родителю: `Bike` и `Car` берут данные и `get_info()` у `Vehicle`, а `move()` каждый реализует сам.

3.1. Абстракция

Абстракция показывает только важное и скрывает детали реализации. Класс `Vehicle` наследуется от `ABC`, поэтому создать объект `Vehicle()` напрямую нельзя. Метод `move()` помечен `@abstractmethod`: каждый наследник обязан его реализовать.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def move(self):
        pass
```

Мы говорим «у любого транспорта есть способ двигаться», но не уточняем какой — это решает каждый вид транспорта.

3.2. Инкапсуляция

Инкапсуляция защищает данные объекта от прямого доступа снаружи. Атрибуты `_brand` и `_speed` начинаются с `_` — это соглашение Python: «внутренние данные, не трогать напрямую». Доступ к ним идёт через метод

`get_info()`.

```
def __init__(self, brand, speed):
    self._brand = brand
    self._speed = speed

def get_info(self):
    return f"{self._brand}, скорость: {self._speed} км/ч"
```

Для более строгой защиты в Python есть двойное подчёркивание (`__speed`) и свойства `@property`.

3.3. Наследование

Наследование позволяет новому классу взять готовый код родителя и не писать его заново. `Bike` и `Car` получают от `Vehicle` атрибуты `_brand`, `_speed` и метод `get_info()`. Вызов `super().__init__()` запускает конструктор родителя.

```
class Bike(Vehicle):
    def __init__(self, brand, speed):
        super().__init__(brand, speed)
```

3.4. Полиморфизм

Полиморфизм — один и тот же вызов метода работает по-разному для разных классов. У `Bike` и `Car` своя версия `move()`, но цикл вызывает её одинаково и не знает, какой именно у него объект.

```
class Bike(Vehicle):
    def move(self):
        return f"{self._brand} едет, крутя педали"

class Car(Vehicle):
    def move(self):
        return f"{self._brand} едет на двигателе"

for v in [Bike("Trek", 25), Car("Toyota", 180)]:
    print(v.move())   # результат зависит от класса объекта
```

4. Полный код программы

Файл `OOP.py` запускается командой `python3 OOP.py`.

```
from abc import ABC, abstractmethod


# 1. АБСТРАКЦИЯ - скрываем сложность, оставляем только нужные методы
class Vehicle(ABC):
    def __init__(self, brand, speed):
        # 2. ИНКАПСУЛЯЦИЯ - данные защищены (приватные через _)
        self._brand = brand
        self._speed = speed

    def get_info(self):
        return f"{self._brand}, скорость: {self._speed} км/ч"

    @abstractmethod
    def move(self):
        pass


# 3. НАСЛЕДОВАНИЕ - Bike наследует от Vehicle
class Bike(Vehicle):
    def __init__(self, brand, speed):
        super().__init__(brand, speed)

    # 4. ПОЛИМОРФИЗМ - свой вариант метода move()
    def move(self):
        return f"{self._brand} едет, крутя педали"


class Car(Vehicle):
    def __init__(self, brand, speed):
        super().__init__(brand, speed)

    # 4. ПОЛИМОРФИЗМ - другой вариант move() для машины
    def move(self):
        return f"{self._brand} едет на двигателе"


# Проверяем работу
vehicles = [Bike("Trek", 25), Car("Toyota", 180)]

for v in vehicles:
    print(v.get_info())
    print(v.move())
    print()
```

Результат работы:

```
Trek, скорость: 25 км/ч
Trek едет, крутя педали

Toyota, скорость: 180 км/ч
Toyota едет на двигателе
```

Попытка создать абстрактный класс напрямую (`Vehicle("X", 10)`) даёт ошибку `TypeError: Can't instantiate abstract class Vehicle`. Так абстракция заставляет работать только с конкретными наследниками.

5. Итоги

Принцип	Что делает	Где в коде
Абстракция	скрывает детали, задаёт общий шаблон	`class Vehicle(ABC)`, `@abstractmethod`
Инкапсуляция	защищает данные, доступ через методы	`self._brand`, `self._speed`, `get_info()`
Наследование	переиспользует код родителя	`class Bike(Vehicle)`, `super().__init__()`
Полиморфизм	одинаковый вызов, разное поведение	разные версии `move()` у `Bike` и `Car`

Вывод. На примере `Vehicle`, `Bike` и `Car` показаны все четыре принципа ООП. Абстракция и наследование убирают дублирование кода, инкапсуляция защищает данные, а полиморфизм позволяет работать с разными объектами одним циклом. Чтобы добавить новый транспорт (например, `Bus`), достаточно написать один класс с методом `move()` — остальной код менять не нужно.